



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Ariel Adara Mercado Martinez

Asignatura: Estructuras de Datos y Algoritmos I

Grupo: 05

No de Práctica(s): 2do Proyecto (PostFija)

Integrante(s): Carrillo Chavira Jorge

García Rayón Dylan Osvaldo

Marín Carrera Adiel Daniel

Arellano Sánchez Brandon Gael

No. de lista o brigada: 01 (Uno)

Semestre: 2026-2

Fecha de entrega: 17/05/2026

Observaciones:

CALIFICACIÓN: _____

Índice

Índice	1
Objetivo	2
Actividades	2
Introducción	2
Desarrollo	3
Pila.c	3
Cola.c	5
evaluator.c	8
parser.c	12
Conclusión Grupal	13
Conclusiones Individuales	14
Jorge Carrillo Chavira	14
Marín Carrera Adiel Daniel	14
García Rayón Dylan Osvaldo	14
Arellano Sánchez Brandon Gael	15
Comentarios	15
Bibliografía	15

PostFija

Objetivo:

Implementar un sistema de conversión y evaluación de expresiones utilizando pilas y colas dinámicas.

Actividades:

Implementar una pila ligada.

Implementar una cola ligada.

Analizar expresiones algebraicas.

Aplicar precedencia y asociatividad de operadores.

Convertir expresiones infijas a postfijas.

Evaluar expresiones postfijas.

Leer datos desde archivos.

Introducción

En el presente reporte se detalla el desarrollo e implementación del segundo MINI “proyecto” de la asignatura, enfocado en el manejo de la notación PostFija. El objetivo principal de esta práctica es construir un sistema robusto capaz de convertir y evaluar expresiones matemáticas utilizando estructuras de datos lineales, específicamente pilas y colas dinámicas. Para lograr el objetivo, el sistema se diseñó a partir de diversos módulos y recursos los cuales fueron desarrollados por los integrantes del equipo, aun que con un orden impuesto por la profesora para que este termine teniendo una coherencia y facilitar el trabajo en equipo. El desarrollo de cada módulo será especificado por cada alumno.

Desarrollo

El desarrollo del proyecto numero 2, el denominado de postfija, estuvo desarrollado por la colaboracion activa de los integrantes del equipo, cada uno encargandose de una parte del codigo, aqui el desarrollo y explicacion de cada uno de ellos.

Se usaron los codigos jorge el cual desarrolló de pila.c y cola.c respectivamente, aqui el desarrollo de estos casos.

Pila.c

```
#include "../include/pila.h"

#include <stdlib.h>

#include <string.h>

Pila* crearPila(){

    Pila* nueva_pila = (Pila*)malloc(sizeof(Pila));

    if(nueva_pila==NULL){

        return NULL;

    } else {

        nueva_pila->tope = NULL;

        return nueva_pila;

    }

}

void push(Pila* p, void* dato, size_t size){

    NodoPila* nuevo = (NodoPila*)malloc(sizeof(NodoPila));

    nuevo->dato = malloc(size);

    memcpy(nuevo->dato, dato, size);
```

```

    nuevo->siguiente = p->tope;

    p->tope = nuevo;
}

void* pop(Pila* p){

    if(pilaVacía(p)){

        return NULL;

    } else {

        NodoPila* tmp = p->tope;

        void* valor = tmp->dato;

        p->tope = p->tope->siguiente;

        free(tmp);

        return valor;

    }

}

int pilaVacía(Pila* p){

    if(p->tope==NULL){

        return 1;

    } else {

        return 0;

    }

}

void destruirPila(Pila* p){

    while(pilaVacía(p)==0){

```

```

        void* d = pop(p);

        free(d);

    }

    free(p);
}

```

jORGE : Para la pila, me enfoqué directamente en los dos problemas que nos pedía resolver el proyecto: manejar los operadores matemáticos y hacer los cálculos finales. Durante la conversión de infija a postfija, la pila es la que hace el trabajo sucio de mantener los operadores (como + o *) "en espera" respetando su jerarquía; si llega un operador de menor peso, saco los de mayor peso que tenía guardados. Luego, en la etapa de evaluación, la usé como mi memoria temporal: iba metiendo los valores de las variables y, cada vez que leía un símbolo matemático, sacaba los dos últimos números de la pila, hacía la operación y metía el resultado de vuelta.

Cola.c

```

#include "../include/cola.h"

#include <stdlib.h>

#include <stdio.h>

#include <string.h>

Cola* crearCola(){

    Cola* nuevoCola = (Cola*)malloc(sizeof(Cola));

    if(nuevoCola==NULL){

        return NULL;

    } else {

        nuevoCola->frente=NULL;

        nuevoCola->final=NULL;
    }
}

```

```

        return nuevoCola;

    }

}

void enqueue(Cola* c, void *dato, size_t size){

    NodoCola* nuevo = (NodoCola*)malloc(sizeof(NodoCola));

    nuevo->dato = malloc(size);

    memcpy(nuevo->dato, dato, size);

    nuevo->siguiente = NULL;

    if((c->frente==NULL) && (c->final==NULL)){

        c->frente=nuevo;

        c->final=nuevo;

    } else {

        c->final->siguiente=nuevo;

        c->final=nuevo;

    }

}

char dequeue(Cola* c){

    if(colaVacia(c)){

        return '\0';

    } else {

        NodoCola* tmp = c->frente;

        char valor = *(char*)(tmp->dato);

        c->frente = c->frente->siguiente;

        if(c->frente==NULL){

```

```

        c->final=NULL;

    }

    free(tmp->dato);

    free(tmp);

    return valor;

}

}

int colaVacia(Cola* c){

    if((c->frente==NULL) && (c->final==NULL)){

        return 1;

    } else {

        return 0;

    }

}

void imprimirCola(Cola* c){

    NodoCola* tmp = c->frente;

    while(tmp!=NULL){

        char v = *(char*)(tmp->dato);

        printf("%c ", v);

        tmp = tmp->siguiente;

    }

    printf("\n");

}

```

```

void destruirCola(Cola* c) {

    while (colaVacia(c) == 0) {

        dequeue(c);

    }

    free(c);

}

```

JORGE : La cola la armé pensando en que iba a ser el "puente" entre la conversión y la evaluación. Su trabajo en el proyecto es almacenar la expresión postfija final ya ordenada. Mientras mi código iba leyendo la expresión original y usando la pila para acomodar los operadores, todo lo que ya estaba listo lo mandaba a formar a la cola con enqueue. De esta forma, la cola se convirtió en mi lista de instrucciones matemáticas de izquierda a derecha.

evaluator.c

```

#include <stdio.h>

#include <stdlib.h>

#include <ctype.h>

#include <math.h>

#include "../include/evaluator.h"

#include "../include/pila.h"

float obtenerValorVariable(

    Variable variables[],

    int total,

    char nombre

) {

    for (int i = 0; i < total; i++) {

```

```

        if (variables[i].nombre == nombre) {

            return variables[i].valor;

        }

    }

    return 0;

}

```

```

float evaluarPostfija(

    Cola* postfija,

    Variable variables[],

    int totalVariables

) {

    Pila* pila = crearPila();

    while (!colaVacía(postfija)) {

        char token = dequeue(postfija);

        if (isalnum(token)) {

            float valor;

            if (isalpha(token)) {

                valor = obtenerValorVariable(

```

```

        variables,

        totalVariables,

        token

    );

}

else {

    valor = token - '0';

}

push(pila, &valor, sizeof(float));

}

else if (esOperador(token)) {

    float b = *(float*)pop(pila);

    float a = *(float*)pop(pila);

    float resultado;

    switch (token) {

        case '+':

            resultado = a + b;

            break;

        case '-':

            resultado = a - b;

            break;

```

```

        case '*':

            resultado = a * b;

            break;

        case '/':

            resultado = a / b;

            break;

        case '^':

            resultado = pow(a, b);

            break;

        default:

            resultado = 0;

    }

    push(pila, &resultado, sizeof(float));

}

}

float final = *(float*)pop(pila);

destruirPila(pila);

return final;

```

```
}
```

(Dylan). Yo me encargué de modificar el archivo de evaluator.c. Lo que realiza este apartado es que en la función evaluarPostfija permite calcular expresiones en notación postfija utilizando una pila dinámica. El algoritmo recorre cada token de la expresión; si encuentra un operando, lo almacena en la pila, y si encuentra un operador, extrae los operandos necesarios, realiza la operación correspondiente y de nueva cuenta guarda el resultado. Finalmente, el valor restante en la pila representa el resultado final de la expresión.

parser.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <ctype.h>
5
6  #include "../include/parser.h"
7  #include "../include/pila.h"
8  #include "../include/cola.h"
9
10 void cargarArchivo(
11     const char* nombreArchivo,
12     Variable variables[],
13     int* totalVariables,
14     char expresion[]
15 ) {
16
17     FILE* archivo =
18         fopen(nombreArchivo, "r");
19
20     if(archivo==NULL){
21
22         printf(
23             "Error al abrir archivo\n"
24         );
25
26         exit(1);
27     }
28
29     char linea[256];
30
31     *totalVariables = 0;
```

```

32
33     while(
34         fgets(
35             linea,
36             sizeof(linea),
37             archivo
38         ) != NULL
39     ){
40
41         if(
42             strchr(
43                 linea,
44                 '='
45             ) != NULL
46         ){
47
48

```

(Gael) En parser.c se realiza la interpretación y separación de la información del archivo para que pueda ser usada en la conversión y expresión de evaluaciones postfijas.

Conclusion Grupal

El desarrollo de este proyecto representó un reto técnico y colaborativo que nos permitió integrar nuestros conocimientos sobre la programación en C y las estructuras de datos. Como equipo, comprendimos de primera mano cómo las abstracciones teóricas, como las pilas y las colas, son fundamentales para resolver problemas algorítmicos complejos como el análisis sintáctico y la jerarquía de operaciones matemáticas.

La modularidad del proyecto nos demostró la importancia de una buena arquitectura de software: mientras el parser interpretaba el archivo, la pila y la cola gestionaban el ordenamiento, y el evaluador calculaba los resultados. Además, superar desafíos como el manejo de apuntadores genéricos para admitir tanto caracteres como valores flotantes sin pérdida de memoria, consolidó nuestra capacidad para diseñar programas eficientes, seguros y dinámicos.

Conclusiones individuales

Jorge Carrillo Chavira

Lo que realmente me costó trabajo y me obligó a investigar en lo que respecta a la función `pila.c` fue darme cuenta de que necesitaba guardar caracteres (los símbolos) en la primera parte, pero números flotantes (los cálculos) en la segunda. Para no tener que programar dos pilas distintas, me aventé a usar apuntadores genéricos (`void*`) y `memcpy`. Fue un dolor de cabeza pelear con la memoria dinámica al principio, pero me salvó la vida porque logré que los nodos de mi pila guardaran simplemente los "bytes" que les mandara, dejando que el archivo principal decidiera cómo interpretarlos.

De ahí en fuera, no fue nada complicado terminar de hacer las otras funciones, por que esto ya lo había hecho en repetidas ocasiones en prácticas anteriores, justamente por ello es que a mí se me encargó eso, por que soy el que más experiencia tiene en estos sistemas por lo que ideal es que yo realizara estos ejercicios y así fue como sucedió la organización del equipo, de acuerdo a quien conoce mejor que tipo de programas.

Marín Carrera Adiel Daniel

En esta práctica, el me costó implementar la recursividad, y profundizar en el análisis de la complejidad temporal de los algoritmos, me llamó mucho la atención cómo una función aparentemente simple, como la inversión de una cadena, escala a una complejidad de $O(n^2)$; la implementación de la suma de naturales y el recorrido de arreglos me resultó bastante natural.

García Rayón Dylan Osvaldo

Sinceramente, a mí que me tocó modificar el archivo de `evaluator.c`,. De ahí en fuera lo demás se me complicó un poco menos, pues estuve horas checando como podía resolver cómo evaluar en notación postfija. Es uno de los mayores retos que me han puesto hasta ahora.

Arellano Sánchez Brandon Gael

Se comprendió el uso de las estructuras lineales con los problemas a partir del pensamiento matemático. Con ello reforcé conocimientos de apuntadores y expresiones matemáticas.

Comentarios

Jorge Carrillo Chavira

Siendo sinceros, el trabajo fue relativamente facil pero injustamente dificil, esto debido a que, lo unico complicado fue entender el concepto, implementarlo en codigo y despues de ello, solo implementarlo a una cola y una pila... lo dificil fue que el trabajo esta hecho en un 80% alrededor por mi compañero Dylan Garcia y yo, Jorge Carrillo, implementamos casi todo, nuestros otros compañeros solo hicieron codigo de hacer por hacer, que estaba mal y tuvimos que corregir, no checaron nisiquiera los requisitos, y ademas, en el reporte no hicieron la parte que les tocaba, solo añadieron “su parte” pero no aportaron nisiquiera a la creacion del reporte en si mismo, a ultima hora, denuevo, mi compañero y yo tuvimos que hacer todo denuevo.

Tengo entendido que un compañero no subio nisiquiera su parte del codigo, eso ya es a evaluar del alumno en cuestion

Bibliografía

- Digitales

Visual Studio Code: Microsoft. (2024). *Visual Studio Code* (Versión 1.87) [Software de computación]. <https://code.visualstudio.com/>

GitHub: GitHub, Inc. (2024). *GitHub* [Plataforma de desarrollo de software]. <https://github.com/>

Google Classroom: Google. (2024). *Google Classroom* [Servicio web educativo]. <https://classroom.google.com/>

Google Docs: Google. (2024). *Google Docs* [Software basado en la nube].
<https://docs.google.com/>

- **Libros**

Cairó, O., & Guardati, S. (2006). *Estructuras de datos* (3ra ed.). McGraw-Hill.
<https://www.mheducation.es>

Deitel, P., & Deitel, H. (2016). *C How to Program* (8va ed.). Pearson.
<https://www.pearson.com>

Joyanes Aguilar, L., & Zahonero Martínez, I. (2008). *Estructuras de datos en Java*. McGraw-Hill. <https://www.mheducation.es>

El lenguaje de programación C. Brian W. Kernighan, Dennis M. Ritchie, segunda edición, USA, Pearson Educación 1991.

Kernighan, B. W., & Ritchie, D. M. (1991). *El lenguaje de programación C*. Prentice Hall. <https://www.pearson.com>

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4ta ed.). Addison-Wesley Professional. <https://algs4.cs.princeton.edu>